Computer Vision

Lecture 3: Image Classification & Linear Models

Nguyen Manh Toan

Swinburne Vietnam

Week 3

Today's Lecture

1. From Features to Classifiers
2. Data & Evaluation
3. Nearest Neighbour
4. Linear Classification
5. Loss Functions
6. Regularization
7. Optimization
8. The Training Loop
9. Wrap-Up

From Features to Classifiers

Where We Left Off

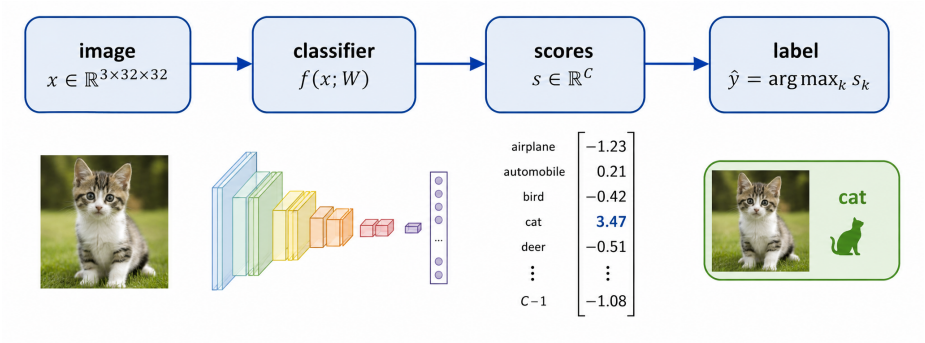
Last week:

- Filters, edges, corners, descriptors are all designed by hand
- Every design choice was really a parameter being fitted to data, slowly, by a human.

This week: let the data set the parameters.

- Define the task precisely, and how to measure success
- The simplest possible classifier (nearest neighbor)
- Linear classifiers: score function, loss function, optimization
- Train a model with gradient descent

Image Classification

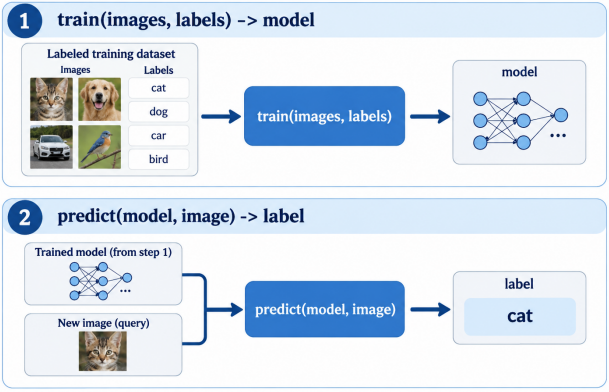


- Input: a fixed-size array of numbers.
- Output: one label from a fixed set of C categories

Everything else — detection, segmentation, captioning — is built on top of this.

The Data-Driven Approach

No obvious algorithm. Cannot write `if` statements over all pixels.
Write a **program that writes the classifier**.



Three core components

What we must supply:

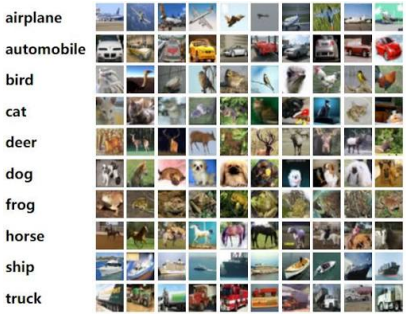
- A **score function** $f(x; W)$ mapping pixels to class scores
- A **loss function** measuring how wrong the scores are
- An **optimization** procedure to find W minimizing the loss

These three pieces define the learning system

Data & Evaluation

Benchmark Datasets

Dataset	Classes	Train	Size	Note
MNIST	10	60,000	28^2 grey	Solved; useful only for debugging
CIFAR-10	10	50,000	32^2 RGB	The teaching standard (figure below)
CIFAR-100	100	50,000	32^2 RGB	Same images, finer labels
ImageNet-1k	1,000	1.28M	$\sim 256^2$	The benchmark that started deep learning



Caution

Benchmark accuracy is not the same as working. Every one of these has known label noise and collection bias

Splitting the Data

- **Train** — the model fits these
- **Validation** — choose hyperparameters on these
- **Test** — touched **once**, at the very end

Typical split: 70 / 15 / 15, or a fixed test set supplied with the benchmark.

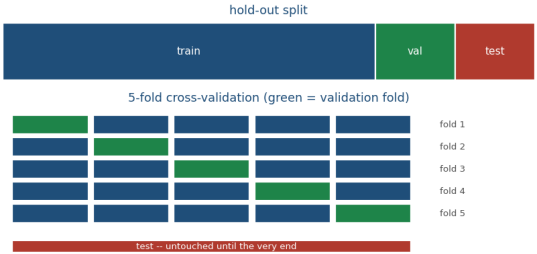
Don't break the rule

Looking at the test set and changing something leaks information into the model.

Cross-Validation

Split the training data into k folds. Train on $k-1$, validate on the remaining one, rotate, and average.

- Uses all the data for both roles
- Gives an **error bar**, not just a number. Can tell a real improvement from noise
- Costs k training runs



In practice

Cross-Validation is standard for small datasets and classical methods, but rare in deep learning. Useful for small projects.

Measuring Performance

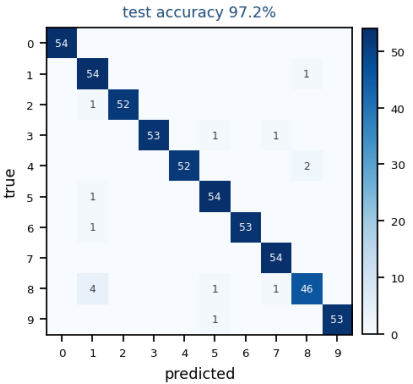
Accuracy $= \frac{\text{correct}}{\text{total}}$ — easy to read, easy to abuse.

- Useless under class imbalance: 99% of scans are healthy, so "always healthy" scores 99%
- Reports nothing about *which* classes fail

Confusion matrix M_{ij}: true class i predicted as j.

- The off-diagonal tells what the model actually confuses
- Almost always the first thing to plot when a model underperforms

Top-5 accuracy: correct if the true label is among the 5 highest scores.



Confusion matrix, linear classifier on MNIST

Nearest Neighbour

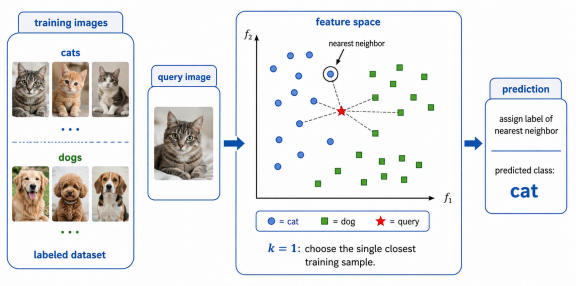
The Simplest Possible Classifier

Train: memorize every training image and its label
Predict: find the most similar training image; copy its label.

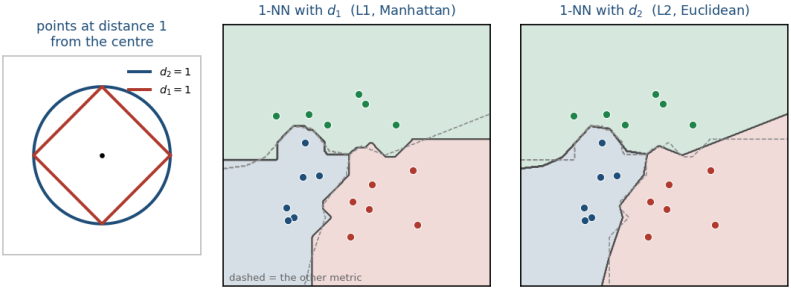
Complexity:

- Train: $O(1)$ — just store the data
- Predict: $O(N)$ — compare against everything

Exactly backward from what we want. Training can take hours, but prediction should be fast.



Choosing Distance



Distance between images, treated as vectors in $\mathbb{R}^D$:

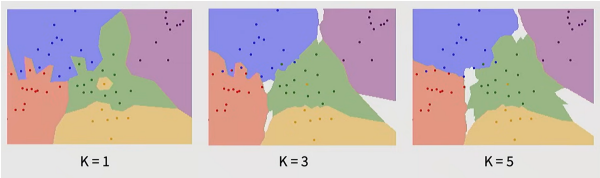
$$d_1(I_1, I_2) = \sum_p \left| I_1^p - I_2^p \right| \qquad (L_1)$$

$$d_2(I_1, I_2) = \sqrt{\sum_p \left(I_1^p - I_2^p \right)^2} \qquad (L_2)$$

k-Nearest Neighbours

Instead of the single closest point, take majority vote from the k closest points.

- $k = 1$: jagged boundary, islands of noise, zero training error
- Larger k: smoother boundary, more robust, but small classes get outvoted
- k and the distance metric are **hyperparameters** — chosen on validation data
- Regions with no clear majority are genuinely ambiguous



Decision boundaries for $k = 1, 3, 5$

Why k-NN Fails on Images

Pixel distance measures *photometric* difference, not semantic difference.

Very different corruptions, nearly identical L_2 distance

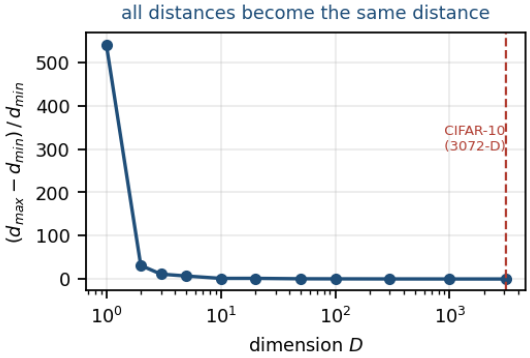
This is the semantic gap from Lecture 1, now with a number attached to it.

The Curse of Dimensionality

Cover $[0,1]^D$ at resolution r need $\sim (1/r)^D$ points.

- $D = 1$, $r = 0.1$: 10 points
- $D = 3$: 1,000 points
- $D = 3072$ (CIFAR-10): more points than atoms in the universe

In high dimensions, the nearest and farthest neighbors become nearly equidistant, so "nearest" stops meaning anything.



$(d_{\max} - d_{\min})/d_{\min}$ **collapses as** D **grows**

Verdict on k-NN

Never use k-NN on raw pixels.

k-NN on **learned features** is everywhere:

- Face recognition: nearest neighbor in embedding space
- Image retrieval, deduplication, RAG
- Few-shot learning

The space matters more than the classifier.

Linear Classification

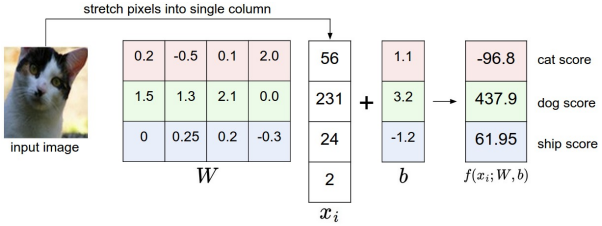
The Score Function

$$f(x_i; W, b) = Wx_i + b$$

With C classes and D-dimensional inputs:

- $x_i \in \mathbb{R}^{D \times 1}$ — one image, flattened
- $W \in \mathbb{R}^{C \times D}$ — the **weights**
- $b \in \mathbb{R}^{C \times 1}$ — the **bias**
- Output $s \in \mathbb{R}^{C \times 1}$ — one score per class

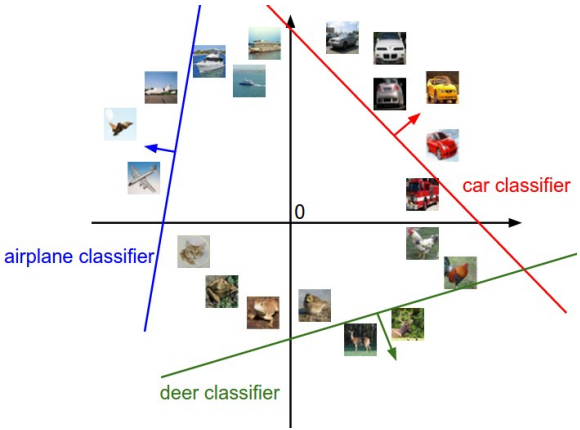
For CIFAR-10: W is 10×3072, b is 10×1 — **30,730 parameters**.



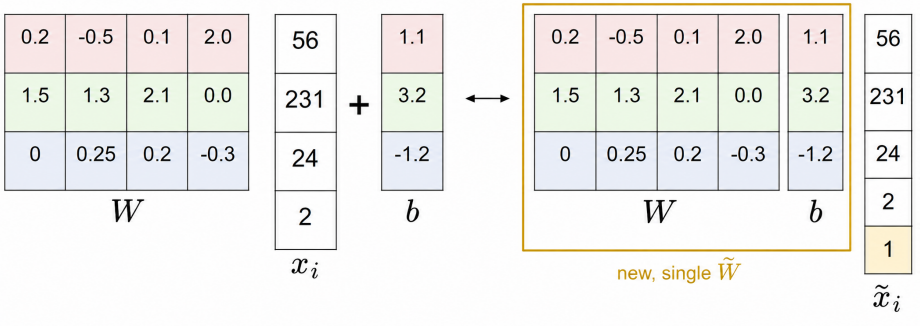
The Score Function

$$f(x_i; W, b) = Wx_i + b$$

- Each row of W is a *classifier for one class*, run in parallel with all the others.
- The blue line shows all points in the space that get a score of zero for the airplane class



The Bias Trick



$$f(x_i) = Wx_i + b = \tilde{W}\tilde{x}_i \leftarrow \text{One matrix, one gradient}$$

In code, usually not

Deep learning frameworks keep `weight` and `bias` separate, because they are often regularized differently — weight decay is applied to W but typically *not* to b.

Interpretation 1: Template Matching

- Row k of W has the same shape as an image.
- The score

$$s_k = w_k^\top x$$

  is an **inner product**: how well the image aligns with template w_k.
- Reshape w_k back to the image shape to see what the classifier learned
- A linear classifier gets **one template per class** — a fundamental limitation: a cat facing left and a cat facing right must share a single template.

Learned weight vectors, reshaped to images

Interpretation 2: Geometry

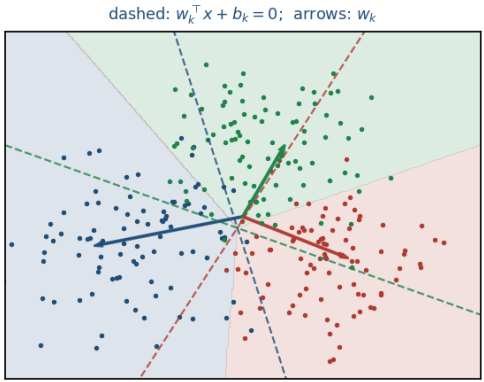
Each image is a point in $\mathbb{R}^D$. For class k:

$$w_k^\top x + b_k = 0$$

is a **hyperplane**. On one side the score is positive, on the other negative.

- w_k sets the orientation
- b_k shifts the plane off the origin
- Changing w_k rotates the boundary; scaling w_k steepens the scores without moving it

Prediction is: which side of which plane, and by how much.



Three linear boundaries in 2D, with score contours

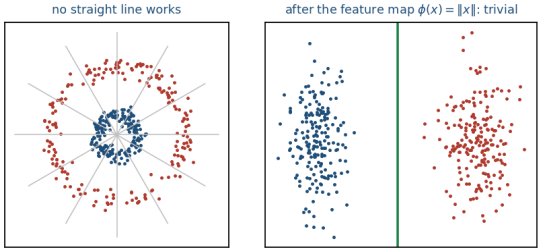
What a Linear Classifier Cannot Do

A single hyperplane can only separate data that is **linearly separable**.

- XOR-like structure: impossible, at any W
- Two concentric rings: impossible
- Multimodal classes (a horse from two viewpoints): one template must cover both, so it covers neither well

Two ways out:

1. Transform the features first, then apply a linear model — this is HOG + SVM
2. **Learn** the transformation — this is a neural network (Week 4)



No line separates these; a learned feature map does

Preprocessing

Mean subtraction — centre the data:

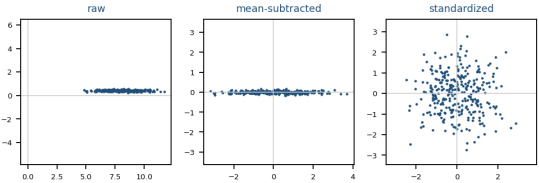
$$x \leftarrow x - \mu, \qquad \mu = \frac{1}{N}\sum_i x_i$$

Normalization — put features on comparable scales:

$$x \leftarrow (x - \mu)/\sigma$$

where σ is the standard deviation.

- Uncentred data makes the loss surface a long narrow valley — gradient descent zig-zags
- Compute μ, σ on the **training set only**, then apply the same numbers to val and test



Raw, centred, and standardized data with the same optimizer

Loss Functions

What Is a Loss?

A loss function turns "these scores are bad" into a number we can minimize.

$$L = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i\big(f(x_i; W),\, y_i\big)}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization}}$$

- $L_i \geq 0$, and $L_i = 0$ exactly when the prediction is as good as we ask
- Must be **differentiable** (almost everywhere) for gradient descent to work

Not accuracy

Accuracy is what we care about, but it is piecewise constant, so gradient is zero everywhere. We optimize a differentiable *surrogate* and hope accuracy follows.

Multiclass SVM Loss (Hinge)

For training sample (x_i, y_i):

$$L_i = \sum_{j \neq y_i} \max\left(0,\ s_j - s_{y_i} + \Delta\right),$$

$$\text{where } s_j = w_j^T x_i + b_j, \quad \Delta = 1$$

- If $s_{y_i} \geq s_j + \Delta$, that term contributes **zero** — already good enough
- Otherwise it contributes linearly in how short we fell
- "Hinge" because of the kink at zero

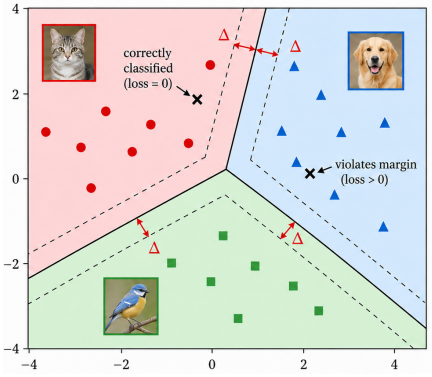


Figure: Multi-class SVM decision regions. —— denotes the decision boundary, while - - - - denotes the margin.

Hinge Loss: Example

Three classes, correct class is the first. $\Delta = 1$.

Case A: $s = [13,\ -7,\ 11]$, $y_i = 0$

$$\begin{aligned} L_i &= \max(0, -7 - 13 + 1) \\ &\quad + \max(0, 11 - 13 + 1) \\ &= \max(0, -19) + \max(0, -1) \\ &= 0 + 0 = 0 \end{aligned}$$

Correct *and* beyond the margin. Nothing to fix.

Case B: $s = [13,\ -7,\ 12.5]$, $y_i = 0$

$$\begin{aligned} L_i &= \max(0, -7 - 13 + 1) \\ &\quad + \max(0, 12.5 - 13 + 1) \\ &= 0 + 0.5 = 0.5 \end{aligned}$$

Still *classified* correctly — but too close. The loss is nonzero, so training continues.

Zero loss is possible while the model is still improvable, and nonzero loss is possible at 100% accuracy. Loss and accuracy are not the same thing.

Binary Classification: Sigmoid and Binary Cross-Entropy

Two classes. One score is enough — "how much do I believe *yes*?"

$$s \in \mathbb{R}, \qquad p = \sigma(s) = \frac{1}{1 + e^{-s}} = P(y = 1 \mid x), \qquad P(y = 0 \mid x) = 1 - p$$

Maximizing the likelihood of the observed label gives the **binary cross-entropy**:

$$L = -\Big[\, y \log p \;+\; (1 - y) \log(1 - p) \,\Big], \qquad y \in \{0, 1\}$$

- Only one term survives: $-\log p$ if $y = 1$, $-\log(1 - p)$ if $y = 0$
- With a linear score $s = w^\top x + b$, this *is* logistic regression
- **One** output unit, not two — the second probability is $1 - p$

Multiclass: Softmax and Cross-Entropy

C classes, **exactly one** correct. Interpret the scores as unnormalized log-probabilities:

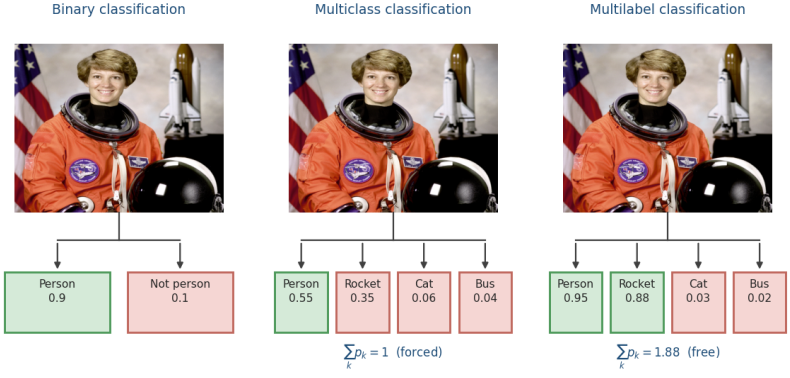
$$P(Y = k \mid X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}} \;=\; p_k \qquad \text{(the \textbf{softmax} function)}$$

Maximize the likelihood of the correct label $\Rightarrow$ minimize its negative log:

$$L_i = -\log p_{y_i} = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right) = -s_{y_i} + \log \sum_j e^{s_j}$$

- $p_k \in (0, 1)$ and $\sum_k p_k = 1$
- The cross-entropy $H(q, p) = -\sum_k q_k \log p_k$ between the one-hot truth q and the prediction p
- At $C = 2$ this reduces exactly to the previous slide, and again $\partial L / \partial s_k = p_k - y_k$

But How Many Objects Are in the Picture?



- One image, one model, three different questions — and three different output layers
- The scene genuinely contains a person **and** a rocket

- Multiclass must *split* its one unit of probability between them: 0.55 and 0.35
- Multilabel can be confident about both: 0.95 and 0.88

Multi-Label: Independent Sigmoids

When several classes can be true at once, drop the constraint $\sum_k p_k = 1$ and ask C separate yes/no questions — binary classification, applied C times:

$$p_k = \sigma(s_k) = \frac{1}{1 + e^{-s_k}}, \qquad L = -\sum_{k=1}^{C} \Big[y_k \log p_k + (1 - y_k) \log(1 - p_k) \Big]$$

- Label is a **binary vector** $y \in \{0, 1\}^C$, not an integer
- No competition between scores — `nn.BCEWithLogitsLoss`
- Predict by thresholding each p_k (usually at 0.5) — there is no `argmax`

Where you will meet it: VOC/COCO classification, scene tagging, medical findings. Detection and segmentation dodge the issue — they classify each *box* or *pixel*, single-label again (Weeks 7–9).

Example: The Same Image, Three Losses

Classes [person, rocket, cat, bus].

Binary — "is there a person?"

$$s = 2.2$$

$$p = \sigma(2.2) = \frac{1}{1+e^{-2.2}} = 0.900$$

Truth $y = 1$:

$$L = -\ln 0.900 = 0.105$$

If the truth were $y = 0$:
$L = -\ln(1 - 0.900) = 2.305$

Multiclass — softmax

$$s = [2.40,\ 1.95,\ 0.18,\ -0.22]$$

$$e^s = [11.02,\ 7.03,\ 1.20,\ 0.80]$$

$$\sum_j e^{s_j} = 20.05$$

$$p = [0.55,\ 0.35,\ 0.06,\ 0.04]$$

Truth = person:

$$L = -\ln 0.55 = 0.598$$

Multi-label — sigmoid

$$s = [2.9,\ 2.0,\ -3.5,\ -3.9]$$

$$p = \sigma(s) = [0.95,\ 0.88,\ 0.03,\ 0.02]$$

Truth $y = [1, 1, 0, 0]$, per class:

$$[0.054,\ 0.127,\ 0.030,\ 0.020]$$

$$L = \sum_k = 0.230$$

Note the middle column sums to 1 by construction; the right-hand one sums to 1.88, and nothing is wrong.

Softmax: Numerical Stability

e^s overflows fast. However,

$$p_k = \frac{e^{s_k}}{\sum_j e^{s_j}} = \frac{e^{s_k - \max_j s_j}}{\sum_j e^{s_j - \max_j s_j}}$$

- Now the largest exponent is $e^0 = 1$: no overflow
- Underflow to zero in the small terms is harmless
- **Always** shift by the max before exponentiating

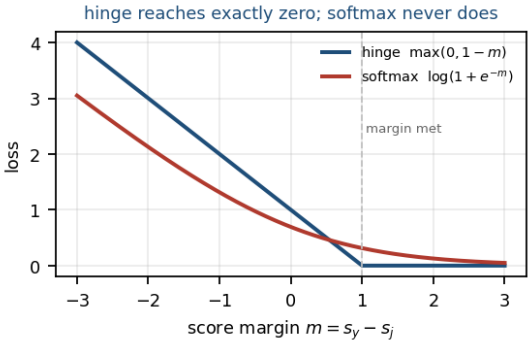
In Pytorch

`torch.nn.CrossEntropyLoss` takes *raw scores*, not probabilities, precisely so it can do this internally. Applying softmax yourself first is a common and quiet bug.

Hinge vs. Softmax

	Hinge	Softmax
Output	scores	probabilities
Zero loss	reachable	never exactly
Once correct	stops caring	keeps pushing
Outliers	robust	sensitive

- Hinge is **satisfied**: past the margin the gradient is exactly zero
- Softmax is **never satisfied**: it always wants p_{y_i} closer to 1
- Cross-entropy dominates modern work because it yields probabilities and composes with everything downstream



Loss as a function of the correct-class score margin

Regularization

W Is Not Unique

If W achieves zero hinge loss, then so does $2W$, and $10W$, and $100W$

The data loss alone cannot express a preference $\implies$ add a new term:

$$L = \frac{1}{N}\sum_i L_i \; + \; \lambda R(W), \text{ where} \qquad R(W) = \sum_k \sum_d W_{k,d}^2 \qquad (L_2)$$

Penalizes large weights. Prefers small, **diffuse** weights over large, concentrated ones.

Why diffuse is better

$x = [1, 1, 1, 1]$, $w_a = [1, 0, 0, 0]$, $w_b = [.25, .25, .25, .25]$.

Both give $w^\top x = 1$, but $R(w_a) = 1$ and $R(w_b) = 0.25$. L_2 prefers w_b, which *uses all four pixels* instead of betting everything on one.

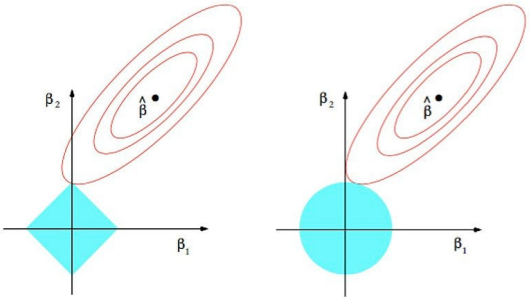
Forms of Regularization

$$R_{L_2}(W) = \sum W^2 \qquad R_{L_1}(W) = \sum |W|$$

$$R_{\text{elastic}}(W) = \sum \left(\beta W^2 + |W|\right)$$

- L_2: shrinks everything smoothly; the default ("weight decay")
- L_1: drives weights exactly to **zero** — gives sparse, selective models
- The corners of the L_1 ball are why it produces exact zeros; the L_2 ball is round and has none

Later: dropout, batch norm, augmentation, early stopping — all regularizers, none a penalty term.



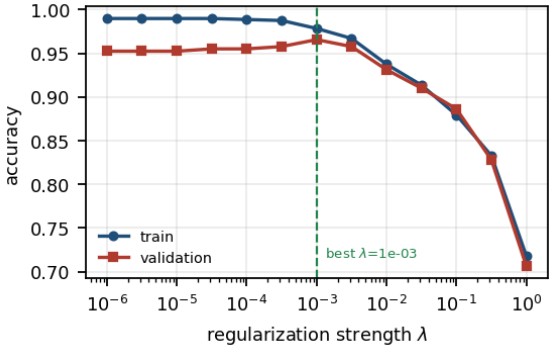
The solid blue areas are the constraint regions (L_1 left, L_2 right) while the red ellipses are the contours of the loss function. L_1 hits the axes, and L_2 does not.

Choosing λ

- λ too small: the model fits the training noise — large train–validation gap
- λ too large: the model cannot fit even the signal — both scores poor
- The best value sits at the minimum of the **validation** curve, not the training one
- Search on a **log scale**: $10^{-6}, 10^{-5}, \ldots, 10^{2}$. Linear grids waste almost all their samples

A Bayesian reading

L_2 is a Gaussian prior on W; L_1 is a Laplace prior. Minimizing loss + penalty is MAP estimation, and λ encodes how strongly we believe the prior.



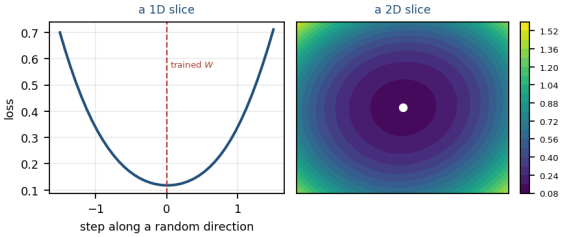
Train and validation accuracy vs. λ

Optimization

The Landscape

$L(W)$ is a scalar function of 30,730 variables. We want its minimum.

- **Strategy 1 — random search.** Try random W, keep the best. It is a genuinely terrible idea
- **Strategy 2 — random local search.** Take a random step, keep it if the loss drops. Better, still hopeless in 30,000 dimensions
- **Strategy 3 — follow the gradient.** The direction of steepest descent is available in closed form, at the cost of one pass



The loss along a random direction in weight space

Gradient Descent

In one dimension:

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

In higher dimensions, the **gradient** $\nabla_W L$ is the vector of partial derivatives.
Update formula:

$$\boxed{W \leftarrow W - \eta \nabla_W L} \qquad \eta = \text{learning rate}$$

$\nabla_W L$ is usually derived *analytically*

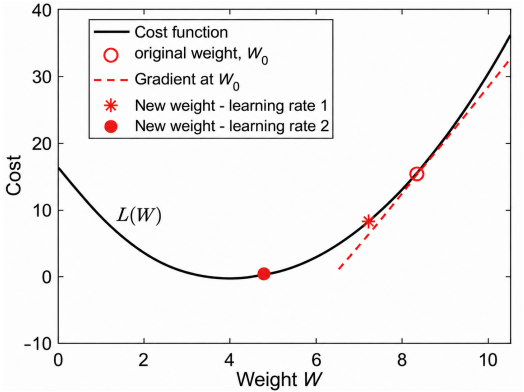


Figure: Gradient points in the direction of steepest *increase*, so we step the other way

Deriving the Softmax Gradient

Let $s = Wx$, $p_k = e^{s_k} / \sum_j e^{s_j}$, and

$$L = -\log p_y = -s_y + \log \sum_j e^{s_j}.$$

Start with $\partial L / \partial s_k$. The first term contributes -1 only when $k = y$. The second term contributes:

$$\frac{\partial}{\partial s_k} \log \sum_j e^{s_j} = \frac{e^{s_k}}{\sum_j e^{s_j}} = p_k$$

Therefore

$$\boxed{\frac{\partial L}{\partial s_k} = p_k - 1[k = y]}$$

From Scores to Weights

We have $\partial L / \partial s$. Since $s = Wx$, the chain rule gives the rest:

$$\frac{\partial s_k}{\partial W_{k,d}} = x_d \qquad \Longrightarrow \qquad \frac{\partial L}{\partial W_{k,d}} = \left(p_k - 1[k = y]\right) x_d$$

In matrix form, averaged over a batch of N examples and with L_2 regularization:

$$\boxed{\nabla_W L = \frac{1}{N} (P - Y)^\top X \; + \; 2\lambda W}$$

where $X \in \mathbb{R}^{N \times D}$, $P \in \mathbb{R}^{N \times C}$ holds the predicted probabilities, and Y is the one-hot matrix of true labels.

Check the shapes:
$(C \times N)(N \times D) = C \times D$, which is the shape of W. Shape-checking catches most gradient bugs.

The Hinge Gradient

For $L_i = \sum_{j \neq y_i} \max(0,\, s_j - s_{y_i} + \Delta)$, let $m_j = 1\big[s_j - s_{y_i} + \Delta > 0\big]$ mark the *violated* margins. Then

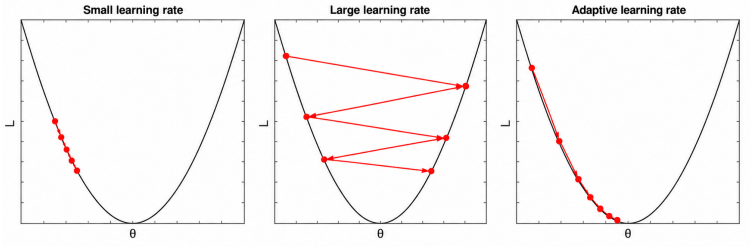
$$\nabla_{w_j} L_i = m_j\, x_i \quad (j \neq y_i), \qquad \nabla_{w_{y_i}} L_i = -\Big(\sum_{j \neq y_i} m_j\Big)\, x_i$$

- Only the classes that *violate* the margin contribute anything
- The correct-class row is pushed up once for every violator
- Examples already past the margin produce **no gradient at all**

Not differentiable at the kink

At exactly $s_j - s_{y_i} + \Delta = 0$ the derivative does not exist. In practice we pick a subgradient (say 0) and move on — hitting the kink exactly has probability zero in floating point.

The Learning Rate



The single most important hyperparameter.

- Too small: correct but glacially slow
- Slightly too large: loss plateaus at a poor value
- Far too large: loss diverges to `nan` within a few iterations

In practice: sweep on a log scale, watch the first few hundred iterations, and pick the largest rate that still decreases smoothly. Then **decay** it over training

Momentum, RMSProp, Adam all refine this (Week 5).

Stochastic Gradient Descent

Computing ∇L over all N examples for *one* parameter update is wasteful. The extreme response: update after every **single** example.

$$W \leftarrow W - \eta \nabla_W L_i, \qquad i \text{ drawn at random}$$

- The estimate is **unbiased**: $\mathbb{E}_i\left[\nabla_W L_i\right] = \nabla_W L$ — right on average, wrong on any given step
- Cost per update drops from $O(N)$ to $O(1)$, so one epoch gives N updates instead of one
- Robbins & Monro, 1951 — older than the field
- Convergence needs a **decaying** η: the noise never dies out on its own

Nobody actually does this

One example at a time cannot fill a GPU

The path is also wildly noisy, so η must be small — cancelling much of the benefit of more steps.

The name survived anyway: what everyone calls "SGD" today is the next slide.

Mini-Batch Gradient Descent

Estimate the gradient from a small random subset B of the data.

$$\nabla_W L \approx \frac{1}{|B|} \sum_{i \in B} \nabla_W L_i, \qquad B \subset \{1, \ldots, N\}, \quad |B| \ll N$$

- Still unbiased, but the variance falls as $1/|B|$
- **Batch size** 32–256 typically; powers of two suit the hardware
- **Iteration** = one parameter update; **epoch** = one full pass, i.e. $N/|B|$ iterations
- Reshuffle every epoch, or the model learns the ordering
- A batch is one matrix multiply — exactly what a GPU is built for

The noise is a feature

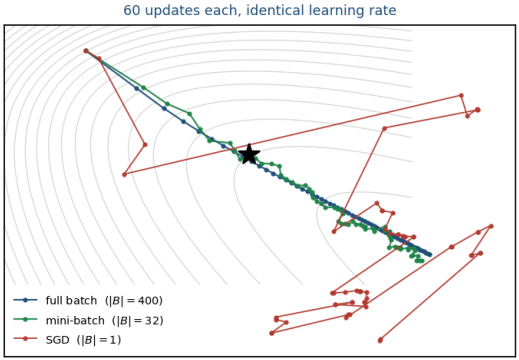
A noisy gradient can escape shallow minima and saddle points that trap full-batch descent. Small batches also generalize better in practice — an effect still not fully explained.

`torch.optim.SGD` + a `DataLoader` batch *is* mini-batch GD.

Full Batch vs. Mini-Batch vs. SGD

	Full	Mini	SGD
$\|B\|$	N	32–256	1
gradient	exact	noisy	very noisy
cost / update	$O(N)$	$O(\|B\|)$	$O(1)$
updates / epoch	1	$N/\|B\|$	N
GPU use	poor	**ideal**	poor

- Full batch is the most *accurate* step and the least *productive* one
- Mini-batch tracks it closely at a fraction of the cost — here the path is only 11% longer
- Pure SGD travels 3.8× further to cover the same ground



At a shared η, $|B|$=1 is visibly unstable — batch size and learning rate must be tuned *together*.

The Training Loop

The Whole Thing, in Few Lines

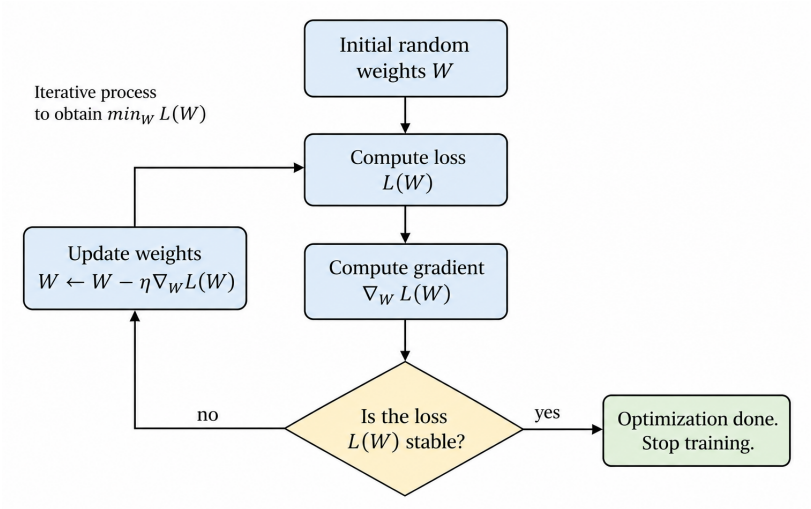
```
W = 0.001 * np.random.randn(C, D)

for it in range(num_iters):
    idx = np.random.choice(N, batch)
    X_b, y_b = X[idx], y[idx]

    scores = X_b @ W.T
    loss, dS = softmax_loss(scores, y_b)
    dW = dS.T @ X_b / batch + 2*reg*W

    W -= lr * dW
```

Every training loop for the rest of this course — ResNet, ViT, a diffusion model — has exactly this shape.

What changes:

- `scores` becomes a deep network
- `dW` comes from autograd instead of your algebra
- `W -= lr*dW` becomes an `optimizer.step()`

Training Loop



Sanity Checks

Before training

1. **Initial loss** $\approx \log C$ for softmax (2.30 at C=10); $\approx C - 1$ for hinge
2. **Turn regularization up** — the loss should go up
3. **Gradient check** on a few random coordinates

During training

4. **Overfit 20 examples** to $\sim$100% accuracy with regularization off. If cannot, the model or the gradient is broken
5. Watch the train–validation gap, not just the loss
6. Plot the loss curve. Its *shape* diagnoses the learning rate

Step 4 costs two minutes and finds most bugs. Skipping it costs a weekend.

The Bridge to Next Week

Today

$$s = Wx$$

- One template per class
- Cannot represent XOR, or a horse from two viewpoints
- Trained on raw pixels, which are a poor representation

Last week's fix: compute better features by hand (HOG), then apply linear model on top.

Next week

$$s = W_2 \max(0,\ W_1 x)$$

- One nonlinearity, and the model can represent XOR
- W_1 *learns* the features; W_2 is still the linear classifier from today
- The loss, the gradient, and the training loop are unchanged

Then we replace the dense W_1 with a **convolution** — and we are back to Lecture 2, with learned kernels.

Wrap-Up

Summary

- **Data-driven classification**: a score function, a loss function, and an optimizer
- k-**NN** is trivial to implement and useless on raw pixels — $O(N)$ at test time, and distance in pixel space does not mean similarity in meaning
- **Linear classifiers** give one template per class; read them three ways — algebra, template matching, hyperplanes
- **Hinge** stops caring once the margin is met; **softmax** never stops. Both need regularization, because W is otherwise not unique
- **The softmax gradient is** $p - y$, and everything else follows by the chain rule
- **SGD** trades an exact gradient for many more steps

Next week: from linear models to neural networks and CNNs.

Questions?

Reading & Reference

- Stanford CS231n notes: *Image Classification*, *Linear Classification*, *Optimization*
- Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. — Ch. 5.1–5.2
- Goodfellow, Bengio & Courville, *Deep Learning* — Ch. 5 (machine learning basics)
- Bishop, *Pattern Recognition and Machine Learning* — Ch. 4 (linear models for classification)

Hands-On (Last 30 Minutes)

Notebook: `lecture03_handson.ipynb`

- Implement k-NN with a vectorized distance matrix; watch the loop version lose by $100\times$
- Tune k by cross-validation, and see the pixel-distance failure directly
- Write the softmax loss — naive with loops, then vectorized — and check they agree
- Derive and implement the analytic gradient; **gradient check** it against the numerical one
- Train with SGD; sweep the learning rate and read the loss curves
- Visualize the learned weight templates
- Overfit 20 examples on purpose, as the sanity check says to
- Reproduce the whole thing in PyTorch with `nn.Linear` and confirm the numbers match